Subiecte

Calcul Lambda

- Aparitii legate vs libere
- O variabila este legată dacă toate aparițiile sunt legate
- Beta reducere (lambda(x).e1 e2) -> e1[e2/x] (aparițiile libere ale lui x **in e1** sunt inlocuite cu e2)
- Alfa-conversie = redenumesc variabile legate a.i. sa nu coincida cu cele libere

Exercitii

Reduceți expresia lambda $(\lambda x.(\lambda x.(x\ \ x)\ \ \lambda x.x)\ \ (x\ \ x))$

1. `(λx y z.corp a b c)` este forma prescurtată de a scrie `(((λx.λy.λz.corp a) b) c)`.

   Fie următoarele definiții în Calcul Lambda:

   `true = λx y.x`

   `false = λx y.y`

   și fie operatorul logic:

   `op = λx y.(x y true)`

   Descrieți pas cu pas (nu efectuați mai multe β-reduceri deodată) comportamentul lui `op` pe valori de tip `true` sau/și `false`, astfel încât să puteți conchide la final că `op` se comportă ca unul dintre operatorii logici cunoscuți (și precizați care este operatorul a cărui identitate a fost ascunsă prin folosirea numelui "`op`").

1. Încercuiți aparițiile legate și subliniați aparițiile libere ale variabilelor din expresia `(λx.(λy.(x y) λz.(y z)) x)`.

1. Reduceți la forma normală următoarea expresie, ilustrând pașii de reducere: $\lambda x.\lambda y.((\lambda x.\lambda y.x\ \ (y\ \ x))\ \ (x\ \ y))$

1. Determinați forma normală pentru următoarea expresie, ilustrând pașii de reducere: $((\lambda x.\lambda y.\lambda z.(x\ \ y)\ \ \lambda x.x)\ \ z)$

1. Ilustrați cele două posibile secvențe de reducere pentru expresia: $(\lambda y.(\lambda x.\lambda y.x\ \ y)\ \ 2)$

Tipuri de evaluari

- Normala (calcul lambda) - argumentele se evaluează numai atunci cand e nevoie de valoarea lor
- Aplicativa (Racket) - argumentele se evaluează înainte de aplicarea funcției
- ++ in Racket or/and/if sunt exceptii (functii nestricte)

Exercitii

1. Folosind evaluare normală, aduceți la forma normală expresia `(λx.(λy.(x y) λz.(y z)) x)`.

Let-uri

- Racket
  - let
  - let*
  - letrec
  - named let
  - ++ define
- Haskell
  - let
  - where

Exercitii

2. Care este diferența între următoarele două linii de cod Racket

```
(let ((a 1) (b 2)) (let ((b 3) (c (+ b 2))) (+ a b c)))
(let* ((a 1) (b 2)) (let* ((b 3) (c (+ b 2))) (+ a b c)))
```

3. Ce întoarce următoarea expresie în Racket? Justificați!

```
(letrec ((f (lambda (n)
  (let ((n (- n 1)))
    (if (eq?  n -1) 1 (* (+ n 1) (f n)))))))
  (f 5))
)
```

12. Explicați care dintre următoarele apeluri dă eroare și care nu, și justificați pentru fiecare:
    1. `(if #t 5 (/ 2 0))` (Racket)
    2. `(let ((f (λ (x y) x))) (f 5 (/ 2 0)))` (Racket)
    3. `let f x y = x in f 5 (div 2 0)` (Haskell)
    4. `X = 2 / 0, Y = X.` (Prolog)

Inchideri functionale ++ evaluari

- La liste, quote(‘) evita evaluarea

Exercitii

```
Varianta 4
----------

Fie programul Haskell de mai jos:

lst = [(1+2, 3+4), (3+4, 5+6), (5+6, 7+8)]

find ((7,x) : _) = x
find (_ : pairs) = find pairs

Descrieți cum decurge, pas cu pas, evaluarea expresiei (find lst).
```

4. Cum decurge, pas cu pas, evaluarea expresiei:

[redacted]

```
head $ ([1] ++ [2]) ++ [3]
```

presupunând cunoscută implementarea operatorului de concatenare:

```
[] ++ ys = ys
(x : xs) ++ ys = x : (xs ++ ys)
```

5. (a) Câți pași de concatenare sunt realizați pentru evaluarea expresiei Racket `(car (append '(1 2) '(3 4)))` ?
   (b) Dar pentru expresia Haskell `head $ [1, 2] ++ [3, 4]` ?

1. Fie definiția și aplicația funcției Racket de mai jos. Descrieți **pas cu pas** cum decurge evaluarea aplicației, utilizând modelul bazat pe substituție textuală.

```
1 (define (f x y)
2   (lambda (z)
3     (if (< x 10) (+ x y) z)))
4
5 > ((f (+ 0 1) (+ 2 3)) (+ 4 5))
```

Promisiuni

Exercitii

2. Se dă următorul cod Racket:

```
(define computation (delay (+ 5 5)))
(* 5 5)
(define (f x) (cons x (force computation)))
(map f '(1 2 3 4))
```

   (a) De câte ori se realizează adunarea?
   (b) Prima evaluare a adunării se realizează înainte sau după înmulțire?
   (c) Rescrieți codul pentru `computation` și pentru `f` folosind închideri funcționale în loc de promisiuni și răspundeți din nou la întrebările de la (a) și (b).

5. Care sunt valorile expresiilor din liniile 6, 7, 11 și 12, din următorul cod Racket?

```
1  (define x 0)
2
3  (define closure (lambda () x))
4  (define promise (delay x))
5
6  (closure)
7  (force promise)
8
9  (define x 1)
10
11 (closure)
12 (force promise)
```

6. Fie următoarea definiție și aplicare în linia de comandă a unei funcții în Haskell:

```
1 f x y = x + x
2 > f (1 + 2) (3 + 4)
```

   Rescrieți cele două expresii în Racket, astfel încât să obțineți aceleași efecte ca în Haskell, în privința modalității de evaluare.

Stream-uri

Exercitii

4. Folosind interfața Racket pentru fluxuri, implementați fluxul x, x^2, x^3, x^4 ... etc. **(5p)**
   Folosiți-vă de această implementare pentru a implementa fluxul de liste (2 3), (4 9), (8 27), (16 81) ... etc. **(5p)**

```
Varianta 1
----------

Pornind de la fluxul numerelor naturale din Racket, definiți fluxul
fluxurilor de numere naturale consecutive, din 2 în 2, din 3 în 3
ș.a.m.d. Exemplu: ((0 1 2 3 ...) (0 2 4 6 ...) (0 3 6 9 ...) ...).
Considerați că naturals este deja definit și că puteți utiliza orice
funcționale pe fluxuri, discutate la curs și laborator.

Hint: În loc să construiți de la zero fiecare flux interior,
gândiți-vă cum îl puteți obține pe baza fluxului anterior.
```

4. Se dau în Racket două fluxuri de liste, `s1` și `s2`. Definiți fluxul ale căror elemente se obțin prin intersectarea listelor de pe aceeași poziție din `s1` și `s2`.

   Exemplu: `s1 = '((1 2) (3 4 5) ...), s2 = '((2 3) (1 4 5) ...), rezultat = '((2) (4 5) ...)`

Functii curry vs uncurry ++ functionale

- Map, filter, apply, foldl, foldr

- Racket vs haskell diferente

Exercitii

3. În Racket, **fără a folosi recursivitate explicită**, scrieți o funcție **curry** care primește o listă de funcții și o listă de argumente și întoarce lista rezultatelor aplicării fiecărei funcții din prima listă pe argumentul de pe poziția corespunzătoare în a doua listă. (prima funcție pe primul argument, a doua pe al doilea, etc.) **(7p)**

Apelați corespunzător această funcție pe o primă listă care conține funcțiile list, odd? și null?, respectiv o a doua listă care conține valorile '(1), 2 și '(3) **(2p)** și precizați ce se obține în urma apelului. **(1p)**

Definiți, în Racket, funcția compare-seq care primește o listă numerică L și întoarce o listă de elemente de tip 'LT, 'EQ sau 'GT după relația de ordine între elementele consecutive ale listei L ('LT atunci când primul e mai mic decât al doilea, 'EQ când sunt egale, 'GT când primul e mai mare).

ex:
(compare-seq '(1 1 2 1 4 3)) => '(EQ LT GT LT GT) (explicație: primul EQ descrie relația dintre elementele de pe primele 2 poziții, apoi LT pentru elementele de pe pozițiile 2 și 3, apoi GT pentru elementele de pe pozițiile 3 și 4, etc.)

Recursivitate pe stiva vs coada vs arborescenta

- **Stiva:** trebuie sa ne întoarcem din recursivitate pentru a prelucra rezultatul
- **Coada:** nu trebuie sa retinem informatiile pe stiva si nu trebuie sa revenim pentru a prelucra rezultatul

- **Arborescenta:** minim 2 apeluri recursive executate necondiționat

Exercitii

2. Ce tip de recursivitate are implementarea de mai jos?

```
(define (z x y)
    (if (or (null? x) (null? y))
        '()
        (cons (car x) (cons (car y) (z (cdr y) (cdr x))))))
```

Definiți (tot în Racket) cât mai eficient funcția **zz** care are același efect cu **z**, dar folosește un alt tip de recursivitate.

Justificați pe scurt eficiența soluției voastre.

3. Fie următoarea funcție Racket:

```
1 (define (f x)
2   (if (<= x 0) 0
3       (- (f (- x 1)) 1))) ;  (f (- (f (- x 1)) 1)))
```

(a) Ce fel de recursivitate utilizează funcția?

(b) Ce fel de recursivitate utilizează funcția, dacă înlocuim ultima linie cu expresia comentată?

Sintetizat de tipuri

```
*Main> :t ($)
($) :: (a -> b) -> a -> b
*Main> :t (.)
(.) :: (b -> c) -> (a -> b) -> a -> c
*Main> :t map
map :: (a -> b) -> [a] -> [b]
*Main> :t filter
filter :: (a -> Bool) -> [a] -> [a]
*Main> :t foldl
foldl :: Foldable t => (b -> a -> b) -> b -> t a -> b
*Main> :t foldr
foldr :: Foldable t => (a -> b -> b) -> b -> t a -> b
*Main> :t zipWith
zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
*Main> :t zip
zip :: [a] -> [b] -> [(a, b)]
*Main> :t iterate
iterate :: (a -> a) -> a -> [a]
*Main> :t repeat
repeat :: a -> [a]
*Main> :t id
id :: a -> a
```

* la liste ignorati Foldable t=>t a, puteti sa folositi direct [a](ex: foldr :: (a -> b -> b) -> b -> [a] ->b)

Exercitii

Sintetizați, pas cu pas, tipul următoarei funcții în Haskell:

```
f x y = [tail x ++ fst y, snd y]
```

Atenție! Simpla menționare a tipului final, fără pași intermediari, nu va fi punctată!

5. Sintetizați tipul funcției f (în Haskell): f x y = iterate $ x . y

4. Sintetizați tipul funcției `f` (în Haskell): `f g h l = map (g . h) l`

7. Sintetizați tipul următoarei funcții în Haskell:

```
1 f x y z = foldr (.) id [x, y, z]
```

Tipul operatorului de compunere este

```
1 (.) :: (d -> e) -> (c -> d) -> c -> e
```

Clase + instantieri

- ==: Eq a
- <, <=, …: Ord a
- show: Show a
- [0..10]: Enum a, Num a

Exercitii

6. Fie definițiile următoare ale tipurilor Card (pentru cărți de joc) și Value (pentru valoarea asociată unei cărți):

```
data Card = C2 | C3 | C4 | C5 | C6 | C7 | C8 | C9 | C10 | J | Q | K | A deriving Show
data Value = Only Int | Either Int Int deriving Show
```

Definiți clasa Valuable caracterizată de o singură funcție value care primește o valoare a tipului membru al clasei și întoarce o valoare de tip Value asociată acesteia. **(3p)**

Adăugați tipul Card la clasa Valuable, astfel încât cărților mici (de la 2 la 9) să li se asocieze propria lor valoare, cărților 10, J, Q, K să li se asocieze valoarea 10, iar asului (A) să i se asocieze fie valoarea 1, fie valoarea 11. **(7p)**

6. Instanțiați clasa `Ord` pentru funcții Haskell care iau un argument numeric, astfel încât o funcție este "mai mică" decât alta dacă valoarea ei este mai mică decât a celeilalte funcții **în cel puțin unul** dintre numerele întregi între 1 și 10.

4. În Haskell, funcționala `map` se poate extinde pentru a opera cu funcții parțiale, care întorc `Nothing` dacă nu sunt definite într-un anumit punct, în forma

```
mapMaybe :: (a  -> Maybe b)
         -> [a] -> [b]
```

unde constructorul de tip `Maybe` este definit ca

```
data Maybe a = Nothing | Just a
```

Diferența față de funcționala `map` standard constă în posibilitatea funcționalei `mapMaybe` de a *renunța* la elementele pentru care funcția de aplicat întoarce `Nothing`. De exemplu, expresia de mai jos se va evalua la `[40, 60]`.

```
mapMaybe (\x -> if x == 5
                then Nothing
                else Just $ x * 10)
         [4, 5, 6]
```

Implementați funcționala `mapMaybe`, folosind **exclusiv funcționale**. Rezolvările care nu respectă condiția **NU** vor fi punctate!

FOL. Rezolutie

- Transformare in FNC

- Implicatiile $p \Rightarrow q$ devin $\neg p \ V \ q$
- Negațiile se bagă in paranteze
  - $\neg(p \ V \ q)$ devine $\neg p \ \wedge \neg \ q$
  - $\neg(p \wedge q)$ devine $\neg p \ V \neg \ q$
  - $\neg(\forall x. p(x))$ devine $\exists x. \neg p(x)$
  - $\neg(\exists x. p(x))$ devine $\forall x. \neg p(x)$
- Distribuirea $\wedge$ fata de $\wedge$: $p \wedge (q \ V \ r)$ devine $(p \wedge q) \vee (p \wedge r)$

- Clauzele
  - $p \ V \ q \ V$.. devine {p, q, …}
  - $p \wedge q \wedge$.. devine {p}, {q}, …

Exercitii

7. Demonstrați că {a, a ⇒ b, b ⇒ c} ⊨ c în două moduri: prin rezoluție **(5p)**, respectiv cu tabele de adevăr **(5p)**.

7. Folosiți pentru scriere `NOT()`, `AND` și `OR` ca operatori logici și `ORICARE` și `EXISTĂ` drept cuantificatori. Scrieți cu caps ca să nu existe confuzii cu limbajul obișnuit. Puteți prescurta numele predicatelor.

   Știind că "Cine împarte (din ceva), parte își face (din acel ceva).", și că

   `NOT(își_face_parte_din(marcel, tort))`, dorim să demonstrăm că `NOT(împarte(marcel, tort))`.

   a) Traduceți proverbul în LPOI (FOL) utilizând numai predicatele menționate în problemă.

   b) Demonstrați **prin metoda rezoluției** concluzia cerută.

   Nu se punctează demonstrațiile care nu folosesc metoda rezoluției.

7. Transformați propoziția „Nu tot ce zboară se mănâncă." în logică cu predicate de ordinul întâi.

8. Utilizând metoda rezoluției, demonstrați că:

$$\{\forall x.P(x) \vee \forall x.Q(x)\} \models \forall x.(P(x) \vee Q(x)).$$

Prolog + metapredicate

- Findall, bagof, setof, forall

- Operatorul cut

Exercitii

8. Implementați, în Prolog, predicatul flatten care primește o listă de atomi și/sau liste (cu orice nivel de imbricare) și întoarce o listă care conține toți atomii din lista inițială. (ex: flatten([a,[b,[c],[[d]],[],[e]]], F) => F = [a, b, c, d, e].)

9. Se dă un graf descris prin fapte Prolog de tip nod/1 și arc/2. Folosind un **metapredicat**, determinați lista tuturor perechilor de 2 noduri distincte între care există un drum de lungime 3 (3 arce). Lista **nu va conține duplicate**. Drumul poate conține același nod de mai multe ori, doar nodul inițial și cel final trebuie să fie diferite.

```
Implementați folosind metapredicate (forall, findall, bagof, setof),
și fără recursivitate explicită, predicatul index(+List, -Index) care
primește o listă de elemente și întoarce o listă de perechi, fiecare
pereche fiind formată dintr-un element E din List și numărul de
apariții ale lui E în List. Hint: în Index elementele apar sortate.

Exemplu:
index([8, 1, 2, 3, 1, 2, 5, 4, 7, 6, 2, 5], X) leagă X la [(1, 2),
(2, 3),  (3, 1),  (4, 1),  (5, 2),  (6, 1),  (7, 1),  (8, 1)]
```

9. Pentru implementarea de mai jos, spuneți în câte moduri (și care sunt acestea, din punct de vedere al legării variabilelor) se va satisface scopul **sel(Y, [X, 2, Y, 4], 2, [1, Z, Z, 4]).**

```
sel(X, [X|List], Y, [Y|List]).
sel(X, [X0|XList], Y, [X0|YList]) :- sel(X, XList, Y, YList).
```

9. Fie următorul program Prolog:

```
1 a(1). a(2).
2 b(1). b(2).
3
4 c(X, Y) :- a(X), b(Y).
5 c(3, 4).
6 c(4, 3).
```

Unde ar trebui plasat operatorul `cut`, astfel încât scopul următor să fie satisfăcut?

```
1 ?- findall(_, c(X, Y), L), length(L, 4).
```

7. Implementați în Prolog un predicatul `p(L1, L2, L3)`, care primește în L1 și L2 două liste de lungimi egale și leagă L3 la o listă de perechi, fiecare pereche conținând elementele de la același index din L1 și L2 (similar `zip` din Haskell) . Exemplu: este adevărat `p([1, 2, 3], [a, b, c], [(1, a), (2, b), (3, c)])`.

Problema

Problema 1

6. Fie definițiile următoare ale tipurilor Card (pentru cărți de joc) și Value (pentru valoarea asociată unei cărți):

```
data Card = C2 | C3 | C4 | C5 | C6 | C7 | C8 | C9 | C10 | J | Q | K | A deriving Show
data Value = Only Int | Either Int Int deriving Show
```

Definiți clasa Valuable caracterizată de o singură funcție value care primește o valoare a tipului membru al clasei și întoarce o valoare de tip Value asociată acesteia. **(3p)**

Adăugați tipul Card la clasa Valuable, astfel încât cărților mici (de la 2 la 9) să li se asocieze propria lor valoare, cărților 10, J, Q, K să li se asocieze valoarea 10, iar asului (A) să i se asocieze fie valoarea 1, fie valoarea 11. **(7p)**

10. Vom folosi tipurile Card și Value definite la exercițiul 6 (puteți presupune că exercițiul 6 este deja rezolvat, și că există funcția value care asociază fiecărei cărți valoarea acesteia) pentru a modela un joc de Blackjack. Pentru aceasta, ne interesează doar colecțiile de cărți pentru care suma valorilor cărților este maxim 21.

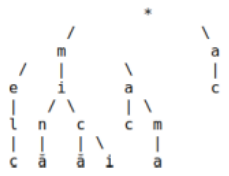
a) Implementați funcția correctValue :: Value -> Maybe Value care primește o valoare de tip Value din care caută să păstreze doar întregii care nu depășesc numărul 21. (ex: correctValue (Either 12 22) => Just (Only 12)) **(10p)**

b) Implementați funcția handValue :: [Card] -> Value care primește o listă de cărți și calculează suma valorilor cărților din listă. Dacă în listă apar unul sau mai mulțiași (A), atunci rezultatul va fi de tip Either suma1 suma2, unde suma1 este calculată pentru cazul când toți așii valorează 1, iar suma2 este calculată pentru cazul când un as valorează 11 și ceilalți valorează 1. (Nu are sens ca mai mai mult de un as să valoreze 11, pentru că suma ar depăși 21). (ex: handValue [A,A,J,C9] => Either 21 31) **(15p)**

c) Implementați **point free** funcția correctHandValue :: [Card] -> Maybe Value care primește o listă de cărți și calculează suma valorilor cărților din listă, cu corecția de la punctul a) deja aplicată, astfel încât să rămână doar sumele utile. (ex: correctHandValue [A,A,J,C9] => Just (Only 21)) **(5p)**

Problema 2

10. Un Trie (un arbore de prefixe) este un arbore care stochează cuvinte, după prefixele lor comune. De exemplu, arborele din dreapta stochează cuvintele "melc", "mină", "mică", "mici", "mac", "mama", și "ac". Fiecare nod, în afară de rădăcină. conține o literă. Rezolvați următoarele cerințe într-un limbaj la alegere dintre Racket, Haskell și Prolog:



a1. descrieți structura de date pentru reprezentarea unui arbore de prefixe (în Haskell, definiți tipul cu `data`, în Racket și Prolog descrieți cum structurați arborele folosind construcțiile oferite de limbaj.
a2. scrieți funcția / predicatul `emptyTrie`, care construiește un arbore de prefixe vid (care nu conține niciun cuvânt).
b1. scrieți funcția / predicatul `canBeFollowedBy` care pentru un caracter și un nod întoarce adevărat dacă nodul are un copil cu valoarea egală cu caracterul dat. În exemplu, pentru primul nod m, funcția întoarce adevărat pentru caracterele e, și și a.
b2. scrieți funcția / predicatul `getSubtreeFor` care pentru un caracter și un nod întoarce nodul copil al nodului dat, care are valoarea egală caracterul dat. E.g. pentru caracterul e și primul nod m din exemplu, funcția întoarce subarborele cu rădăcina în nodul e.
c. scrieți funcția / predicatul `addWord`, care primește un cuvânt și un arbore de prefixe, și adaugă cuvântul la arbore.

Problema 3

10. Problema urmărește reprezentarea unor propoziții din logica propozițională și implementarea unei transformări asupra acestora.

    Utilizați următorul schelet:

    - (8p) Elaborați reprezentarea propozițiilor logice în forma unui tip de date Haskell. Propozițiile vor putea consta în:
      - propoziții simple (exemplu: p, q, r etc.)
      - negații (exemplu: $\neg p$)
      - conjuncții (exemplu: $p \wedge q$)
      - disjuncții (exemplu: $p \vee q$).

      ```
      data Prop
          = Undefined
      ```

    - (2p) Pornind de la reprezentarea de mai sus, traduceți propoziția $\neg(p \wedge (\neg q \vee r))$.

      ```
      prop :: Prop
      prop = undefined
      ```

    - (10p) Instanțiați clasa `Show` cu tipul `Prop`. Reprezentarea sub formă de șir a propoziției prop ar trebui să fie cea din cerința de mai sus.

      ```
      instance Show Prop where
          show = undefined
      ```

    - (10p) Implementați funcția `processNeg`, care urmărește introducerea negațiilor în paranteze. De exemplu:

      `show (processNeg prop)` va furniza $(\neg p \vee (q \wedge \neg r))$

      ```
      processNeg :: Prop -> Prop
      processNeg = undefined
      ```

Solutii

Calcul Lambda

`(op true y) = (λx y.(x y true) true y) → (true y true) = (λx y.x y true) → y`

`(op false y) = (λx y.(x y true) false y) → (false y true) = (λx y.y y true) → true`

Concluzie: `op` este ⇒

1. Încercuiți aparițiile legate și subliniați aparițiile libere ale variabilelor din expresia `(λx. (λy. (x y) λz. (y z)) x)`.

$\lambda x.\lambda y.((\lambda x.\lambda y.x \quad (y \quad x)) \quad (x \quad y)) \rightarrow_\alpha \lambda x.\lambda y.((\lambda x.\lambda z.x \quad (y \quad x)) \quad (x \quad y)) \rightarrow_\beta \lambda x.\lambda y.(\lambda z.(y \ x) \ (x \ y)) \rightarrow_\beta \lambda x.\lambda y.(y \ x)$

$((\lambda \underline{x}.\lambda y.\lambda z.(\underline{x} \ y) \ \lambda x.x) \ z) \rightarrow_\beta (\lambda \underline{y}.\lambda z.(\lambda x.x \ \underline{y}) \ z) \rightarrow_\alpha (\lambda \underline{y}.\lambda w.(\lambda x.x \ \underline{y}) \ z) \rightarrow_\beta \lambda w.(\lambda \underline{x}.\underline{x} \ z) \rightarrow_\beta \lambda w.z$

- $(\lambda \underline{y}.(\lambda x.\lambda y.x \ \underline{y}) \ 2) \xrightarrow{stanga\text{-}dreapta}_\beta (\lambda \underline{x}.\lambda y.\underline{x} \ 2) \rightarrow_\beta \lambda y.2$
- $(\lambda y.(\lambda x.\lambda \underline{y}.x \ y) \ 2) \rightarrow_\alpha (\lambda y.(\lambda \underline{x}.\lambda z.\underline{x} \ y) \ 2) \xrightarrow{dreapta-stanga}_\beta (\lambda y.\lambda z.y \ 2) \rightarrow_\beta \lambda z.2$

Tipuri de evaluari

`E → (λy.(x y) λz.(y z)) → (x λz.(y z))`

Let-uri

În prima definiția `(b 3)` nu este vizibilă în legarea lui `c`; rezultatul este 8, iar în a doua linie rezultatul este 9.

Este factorial. $5! = 120$.

1. Nu este eroare → `if` este funcție nestrictă.
2. Eroare, din cauza evaluării aplicative.
3. Nu este eroare, datorită evaluării lenește (`y` nu este folosit).
4. Nu este eroare, `=` nu evluează calculele aritmetice.

Inchideri functionale ++ evaluari

```
find lst
find ((1+2, 3+4) : ...)
find ((3, 3+4) : ...)
find ((3+4, 5+6) : ...)
find ((7, 5+6) : ...)
5+6
11
```

```
head $ ([1] ++ [2]) ++ [3]       -- (1)
head $ (1 : ([] ++ [2])) ++ [3] -- (2)
head $ 1 : (([] ++ [2]) ++ [3]) -- (3)
1                                -- (4)
```

(a) Se concatenează întregime listele, deci doi pași.
(b) Este suficient un singur pas pentru ca **head** să întoarcă primul element.

```
1 ((f (+ 0 1) (+ 2 3)) (+ 4 5))
2 → ((f 1 5) (+ 4 5))
3 → ((lambda (z) (if (< 1 10) (+ 1 5) z)) (+ 4 5))
4 → ((lambda (z) (if (< 1 10) (+ 1 5) z)) 9)
5 → (if (< 1 10) (+ 1 5) 9)
6 → (if true (+ 1 5) 9)
7 → (+ 1 5)
8 → 6
```

Promisiuni

(a) o singură dată, la prima evaluare a lui `computation`.
(b) după înmulțire, atunci când se apelează prima oară `(force computation)`
(c) `(define computation (λ () (+ 5 5)))`
`(* 5 5)`
`(define (f x) (cons x (computation)))`
`(map f '(1 2 3 4))`
acum se apelează de 4 ori, la fiecare evaluare a lui `computation`; dar prima dată tot după înmulțire.

0, 0, 1, 0. Linia 11 utilizează valoarea calculată în linia 6, la prima forțare (0), în timp ce linia 12 presupune reevaluarea corpului x, care acum este 1. □

```
1  (define (f x y)
2    (+ (force x) (force x)))
3
4  > (f (delay (+ 1 2)) (delay (+ 3 4)))
```

Stream-uri

```
(define (powers x)
  (letrec ((pows (stream-cons x (stream-map (λ (e) (* x e)) pows))))
    pows))
(define f
  (stream-zip-with list (powers 2) (powers 3)))
```

```
(define multiples
  (stream-cons naturals
               (stream-map (λ (s) (stream-zip-with + s naturals))
                           multiples)))

(define result
  (stream-zip-with (lambda (L1 L2) (filter (lambda (e) (member e L1)) L2))
  ↪  s1 s2))
```

Functii curry vs uncurry ++ functionale

```
(define (applications funcs)
  (λ (args)
    (map (λ (f a) (f a)) funcs args)))

((applications (list list odd? null?)) '((1) 2 (3))) => (((1)) #f #f)
```

```
(define (compare-seq L)
  (let* ((l  (length L))
         (L1 (take L (- l 1)))
                ; 2p crearea listei fără ultimul element
         (L2 (cdr L)))
                ; 2p crearea listei fără primul element
                ; 4p aplicarea funcției binare pe cele 2 liste
                ; 2p funcția binară a lui map
    (map (λ (x y) (cond ((= x y) 'EQ)
                        ((< x y) 'LT)
                        ((> x y) 'GT))) L1 L2)))
```

```
(define (zz x y)
    (let z ((x x) (y y) (r '()))
        (if (or (null? x) (null? y))
            (reverse r)
            (z (cdr y) (cdr x) (cons (car y) (cons (car x) r))))))
```

(a) Pe stivă, întrucât se mai realizează operații după evaluarea aplicației recursive.

(b) Tot pe stivă, datorită aplicației recursive interne.

Sintetizat de tipuri

```
f :: a -> b -> c
x :: a
y :: b
tail :: [d] -> [d]
a = [d]
tail x :: [d]
fst :: (e, g) -> e
snd :: (e, g) -> g
b = (e, g)
fst y :: e
snd y :: g
e = [d]
g = [d]   (din aceeași listă cu elemente de tipul e)
f :: [d] -> ([d], [d]) -> [[d]]

x :: a, y :: b, iterate $ x . y :: c, f :: a -> b -> c
iterate :: (d -> d) -> d -> [d]
    => x . y :: d -> d, c = d -> [d]
(.) :: (g -> h) -> (e -> g) -> (e -> h)
    => x :: g -> h, y :: e -> g, x . y :: e -> h
    => e = d, h = d, a = g -> d, b = d -> g
=> f :: (g -> d) -> (d -> g) -> d -> [d]
```

```
f ::  a → b → c → d
map ::  (e → i) → [e] → [i]
l ::  c ⇒ c = [e]
(.)  ::  (t2 → t3) → (t1 → t2) → (t1 → t3)
a = g1 → g2 = t2 → t3
b = h1 → h2 = t1 → t2
g .  h ::  e → i = t1 → t3
c = [e] = [t1] = [h1]
d = [i] = [t3] = [g2]
f ::  (t2 → t3) → (t1 → t2) → [t1] → [t3]
```

```
1 f :: g -> h -> i -> j
2 foldr :: (a -> b -> b) -> b -> [a] -> b
3 a = d -> e  -- foldr + (.)
4 b = c -> d = c -> e  -- foldr + (.)
5 b = k -> k  -- foldr + id
6 c = d = e = k
7 g = h = i = a  -- foldr + lista
8 f :: (k -> k) -> (k -> k) -> (k -> k) -> k -> k
```

Clase + instantieri

```
class Valuable t where
      value :: t -> Value

instance Valuable Card where
      value  A = Either 1 11
      value C2 = Only 2
      value C3 = Only 3
      value C4 = Only 4
      value C5 = Only 5
      value C6 = Only 6
      value C7 = Only 7
      value C8 = Only 8
      value C9 = Only 9
      value  _ = Only 10
```

```
instance (Num a, Enum a, Ord b) => Ord (a -> b) where
  f <= g = or (zipWith (<=) (map f [1..10]) (map g [1..10]))
```

Notă: `Enum a` nu era cerut.

```
1 mapMaybe :: Eq b => (a -> Maybe b) -> [a] -> [b]
2 mapMaybe f = map (\(Just x) -> x) . filter (/= Nothing) . map f
```

FOL. Rezolutie

a) Cele 3 premise în formă clauzală:
(1) {a}
(2) {¬a, b}
(3) {¬b, c}

Rezultate ale aplicării rezoluției:
(4) {b} (din 1, 2)
(5) {c} (din 3, 4)

Obs: se acceptă demonstrat așa sau prin reducere la absurd, introducând ¬c și ajungând la rezolventul vid.

b)

a	b	c	a ⇒ b	b ⇒ c
T	T	T	T	T
T	T	F	T	F
T	F	T	F	T
T	F	F	F	T

....
(rândurile cu a = F oricum nu interesează pentru că nu satisfac premisele)

Rândul marcat este singurul care satisface premisele {a, a ⇒ b, b ⇒ c}, și se observă că acesta satisface și concluzia c.

a) $\forall X.\forall Y.\texttt{împarte(X, Y)} \Rightarrow \texttt{își_face_parte(X, Y)}$

b) Clauzele:

(1) $\neg\texttt{împarte(X, Y)} \vee \texttt{își_face_parte_din(X, Y)}$

(2) $\texttt{împarte(marcel, tort)}$

(3) $\neg\texttt{își_face_parte_din(marcel, tort)}$

(1) + (2) $\{X \leftarrow \texttt{marcel}\} \rightarrow \texttt{își_face_parte_din(marcel, tort)}$ (4)

(3) + (4) $\rightarrow \square$ (clauza vidă)

$\exists x.zboara(x) \wedge \neg se_mananca(x)$ – există și lucruri care zboară și nu se mănâncă
sau
$\neg(\forall x.zboara(x) \Rightarrow se_mananca(x))$ – nu este adevărat că orice care zboară automat se și mănâncă

Pentru claritate, redenumim, variabilele, obținând:

$$\{\forall x.P(x) \vee \forall y.Q(y)\} \models \forall z.(P(z) \vee Q(z)).$$

Utilizăm reducerea la absurd, conform căreia adăugăm negația concluziei la mulțimea de propoziții: $\neg\forall z.(P(z) \vee Q(z)) \equiv \exists z.(\neg P(z) \wedge \neg Q(z))$. Astfel, în urma aplicării procedurii de transformare în forma normală conjunctivă, care presupune și înlocuirea variabilei cuantificate existențial z cu constanta c_z, se obțin clauzele $\{P(x), Q(y)\}, \{\neg P(c_z)\}, \{\neg Q(c_z)\}$. Aplicând rezoluția în raport cu substituția $\{x \leftarrow c_z, y \leftarrow c_z\}$, se obține clauza vidă. $\square$

Prolog + metapredicate

```
flatten([], []).
flatten([[]|Rest], Res) :- !, flatten(Rest, Res).
flatten([[X|R]|Rest], Res) :- !, flatten([X|R], Aux1), flatten(Rest, Aux2), append(Aux1,
Aux2, Res).
flatten([X|Rest], [X|Res]) :- flatten(Rest, Res).
```

```
setof((X,Y), Z^T^(arc(X,Z), arc(Z,T), arc(T,Y), X \= Y), L).
```

```
index(List, Index) :-
      setof((E, Times),
            L^(
            member(E, List),
            findall(X, (member(X, List), E = X), L),
            length(L, Times)
            ),
            Index).
```

sau

```
index(List, Index) :-
      setof(E, member(E, List), Es),
      findall((E, Times),
            (   member(E, Es),
                  findall(X, (member(X, List), E = X), L),
                  length(L, Times)
            ),
            Index).
```

```
2 moduri: Y = Z, Z = 2, X = 1   ;   X = 1, Z = 2   ;   false.
```

```
c(X, Y) :- !, a(X), b(Y).
```

Astfel, se generează doar cele patru combinații între `a` și `b`, renunțându-se la definițiile din liniile 5 și 6. □

```
zip(A, B, C) :- findall((X, Y), (nth0(I, A, X), nth0(I, B, Y)), C).
```

Problema

Problema 1

```
correctValue :: Value -> Maybe Value
correctValue (Only s)
      | s > 21 = Nothing
      | otherwise = Just (Only s)
correctValue (Either s1 s2)
      | s1 > 21 && s2 > 21 = Nothing
      | s1 > 21 = Just (Only s2)
      | s2 > 21 = Just (Only s1)
      | otherwise = Just (Either s1 s2)

handValue :: [Card] -> Value
handValue cards =
      let aces = length $ filter (== A) cards
          others = filter (/= A) cards
          valueOthers = sum [ v | c <- others, let Only v = value c ]
      in if aces == 0 then Only valueOthers
                      else Either (aces + valueOthers) (aces + 10 + valueOthers)

correctHandValue :: [Card] -> Maybe Value
correctHandValue = correctValue . handValue
```

Problema 2

```
data Trie = -- rădăcina are doar copii, nu conține niciun caracter
            Root { getChildren :: [Trie] }
            -- un nod conține un caracter și o listă de copii
            | Node {
                getChar :: Char,
                getChildren :: [Trie]
                }
            deriving (Show, Eq)

-- alternativ
data Trie1 = Node1 { getChar1 :: Maybe Char, getChildren1 :: [Trie1] }
-- sau getChar :: Char, și se pune '*' în rădăcină
-- sau alte soluții


emptyTrie = Root []
```

```haskell
canBeFollowedBy :: Char -> Trie -> Bool
-- iau caracterele din copii și mă uit dacă este vreun copil cu litera
canBeFollowedBy letter node = elem letter (map getChar $ getChildren node)

getSubtreeFor :: Char -> Trie -> Trie
getSubtreeFor letter node = if null selected then undefined else head selected
    where selected = filter ((==letter) . getChar) $ getChildren node

addWord :: [Char] -> Trie -> Trie
-- dacă s-a terminat cuvântul, atunci înseamnă că este deja în trie
addWord [] node = node
addWord (c:word) node
    -- dacă există deja următorul simbol într-un nod copil
    | canBeFollowedBy c node = constr $
                filter ((/= c) . getChar) chd
                ++
                -- adaug în nodul corespunzător lui c sufixul care a mai rămas
                [addWord word $ getSubtreeFor c node]
    -- altfel, adaug un nod nou, ca copil al nodului curent
    | otherwise = constr $ addWord word (Node c []) : chd
    where
        -- abstractizez cazurile
        constr = case node of Node cn _ -> Node cn; otherwise -> Root
        chd = getChildren node
```

Problema 3

```haskell
-- 1.1
data Prop
    = Simple Char
    | Neg Prop
    | And Prop Prop
    | Or Prop Prop
-- 1.2
prop :: Prop
prop = Neg $ Simple 'p' `And` (Neg (Simple 'q') `Or` Simple 'r')
```



```haskell
-- 2
instance Show Prop where
    show (Simple c) = [c]
    show (Neg p) = "~" ++ show p
    show (And p1 p2) = "(" ++ show p1 ++ " ^ " ++ show p2 ++ ")"
    show (Or p1 p2) = "(" ++ show p1 ++ " v " ++ show p2 ++ ")"
-- 3
processNeg :: Prop -> Prop
processNeg p@(Simple _) = p
processNeg (Neg p) = case p of
    Simple _  -> Neg p
    Neg p'    -> processNeg p'
    And p1 p2 -> processNeg $ Or (Neg p1) (Neg p2)
    Or p1 p2  -> processNeg $ And (Neg p1) (Neg p2)
processNeg (And p1 p2) = And (processNeg p1) (processNeg p2)
processNeg (Or p1 p2) = Or (processNeg p1) (processNeg p2)
```